SRA → PARu
MBRA → SIMRu

Rational Agent

→ makes informed decisions like a person, firm, s/w, machine

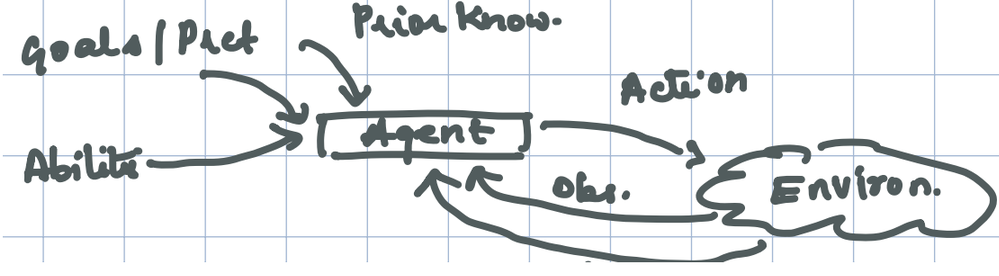
→ Carries out the best outcome based on its past & current percepts.

AI System = Agent + Environment ↙ contains mult. agents
↳ acts on the environ.

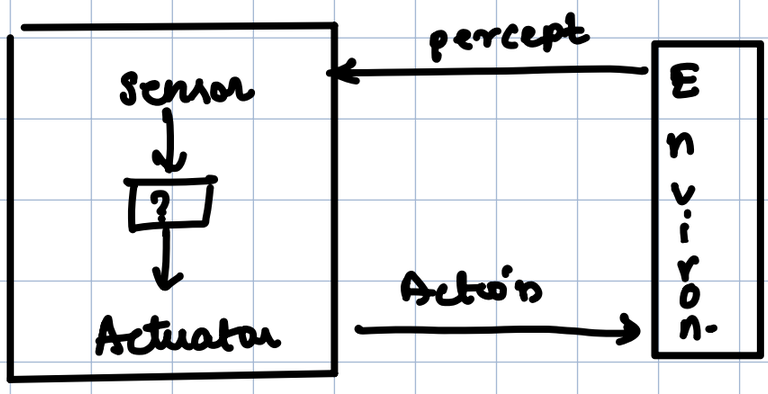
Agent:
- percieves the environment through sensors
- acts upon " " " actuators

ARCHITECTURE = SENSOR + ACTUATOR
AGENT = ARCH. + PROGRAM

eg: Human agent → Sensor: Eyes, nose, ears etc
Actuators: muscle, legs, voice



past exp.



① Percept : a perceptual i/p at any instant

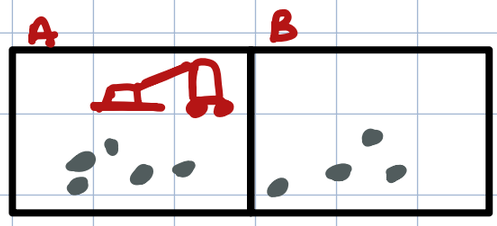
- Agent's PERCEPT SEQ. = complete history of everything perceived.

- Agent's choice of action maybe → Per. Seq. but not anything it hasn't perceived.

② Agent function : maps any given percept seq to action

③ Agent program: the code that implements the agent funcn. for an AI agent.

VACUUM CLEANER PROBLEM



Problem Formulation

Initial State → State where the problem starts
A/B any state could be initial

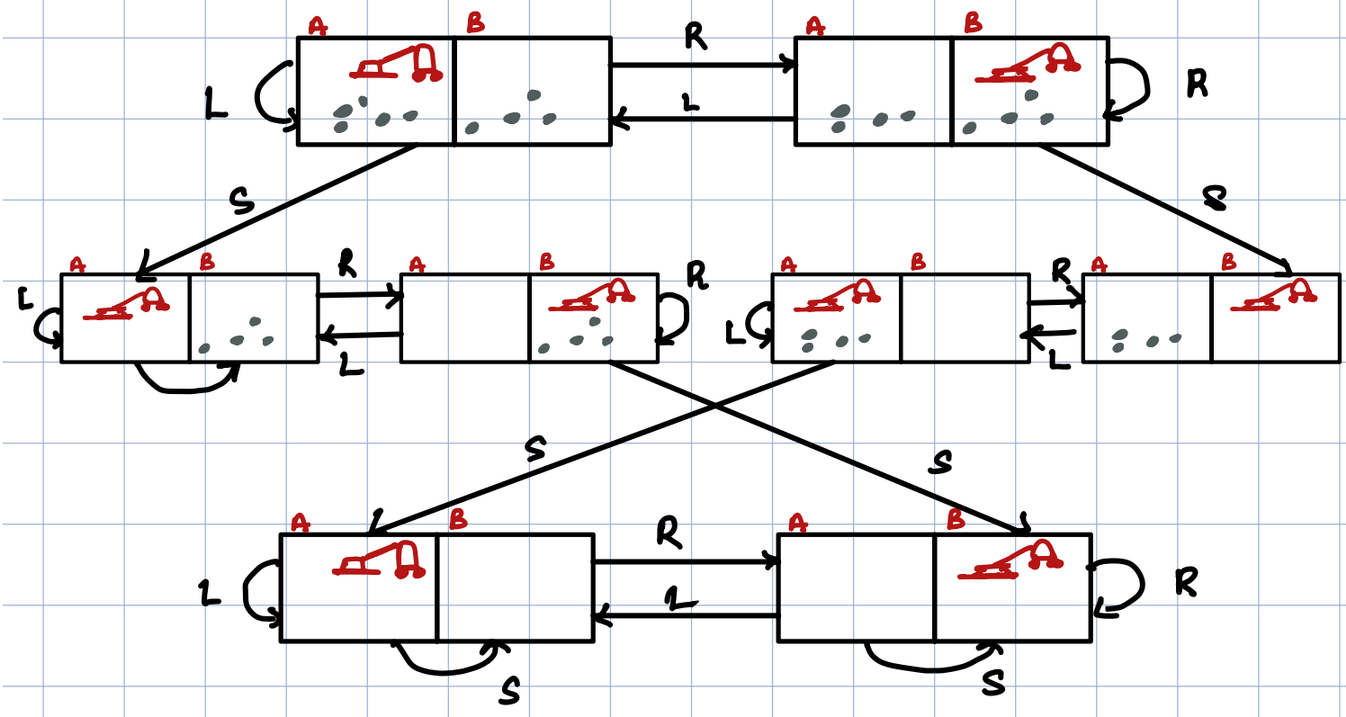
Actions → what are the actions agent could take
Move Left, Move Right, Suck

function → Result (s, a)
s ← state
a ← action

Goal State → Both rooms A/B are clean

Path cost → Each step costs = 1
∴ path cost = no. of steps

State Space / Transition



PEAS

① P : Both rooms clean, time taken, min energy consumption

② E : Two rooms (Clean / Dirty)

③ A : Motors for moving, suction tube to suck dirt, wheels

④ S : dirt sensor, location sensor, bumpers

Rationality of an agent depends on:

① Performance Measure → define the criteria for success.

② Agent's prior knowledge of environment

③ The action that it can perform

④ The agent's percept seq. upto date.

Properties it should have — Gather info
— Learn from the exp
— autonomy
— augm. knowledge

PEAS → type of model on which AI agents work.

① Performance Measure : Criteria to judge the success / perf. of the agent
↳ how do we judge if the agent is doing well — eff — cost — comf — safety — accuracy — speed

② Environment: the real environment where the agent needs to deliberate its actions.

③ Actuators : tools / equipments that agents use to perform actions on the env.
o/p of the agent

④ Sensors: tools / equipments that capture the state or percieve the environment.
I/p to the agent

Example

P → what qualities (business / client)
E → what is its surrounding
A → how will it perform action S → Sense

System	PEAS
Auto-door OS	P : efficiency time taken, power consumed, noise E : Area on both sides of the door A : Motors closing & opening " " S : motion detection, camera on both sid
Face Auth System	P : accuracy, time taken, ab. to handle new E : face detected by camera A : authentication response S : camera / sensor sensing presence
Automated Car Driver	P : efficiency in navigation, safety, optimum speed E : Roads, traffic, weather A : steering wheel, AB, gears

S : cameras , lidar , object detection

Task Environment → refers to the actions, outcomes & choices a given user has for a given task.

* diff b/w task env & env ← world
↓
world struct for the spec. task.

Properties (P):

→ Fully v/s Partially Observable
→ Single v/s Multi Agent
→ Deterministic v/s Stochastic
→ Episodic v/s Sequential
→ Static v/s Dynamic
→ Discrete v/s Continuous
→ known v/s Unknown.

①

Fully Observable - agent's sensors give access to the complete state of the env. at each point in time.

↳ easy because no need for internal state to keep track history of the world.

Unobservable - agent has no sensors at all to the environment

* Env = P.O ⇒ noise + inaccurate sensors / parts of state are simply missing

Eg:

FO → Chess (Board full visible + moves of opp)

PO → Driving (Blind corners + weather)

Pokers (Opp. cards not known)

Angry Birds - FO Minecraft / Temple Run - PO

← ask randomness involved in one action multiple outcomes

Deterministic – next state of envi completely depends on current state + action of the agent

↳ In practical situations → it is impossible to keep track of all aspects of a state ie unobserved aspects ⇒ STOCHASTIC

Eg: Driving → stochastic
Vacuum → Deterministic

← simpler because no need to think ahead

Episodic - agent's exp. is divided into atomic episodes
↳ each ep = a single action
ie no past + no future effects

eg: Defect detection on part on assembly line

Sequential – current decision may affect all future decisions.
eg: Chess, Driving

Static v/s Dynamic

↓

environment can change while an agent is deliberating ⇒ Football

easy cause while taking a decision need not check the world or worry abt time.

↓

mowing lawn

Discrete v/s Continuous

↳ applies to 3 parts ① state of env.
② way time is handled
③ action + percept

Chess : → state : FINITE set of distinct states
time : exclude the clock
action + percept : set of discrete steps

Driving: continuous state + continuous time

Known → outcomes for actions are given ← chess

Unknown → agent learns how to work in order to make good decisions

↓

max score

Single Agent – only one agent involved in env operating by itself. ← crossword

Multi-Agent – multiple agents. → chess

driving partially cooperative

Kinds of Agents:

1. Simple Reflex
2. Model-Based Reflex
3. Goal Based
4. Utility
5. Learning Agents

Table Driven Agent

* look up table has actions mapped to certain seq.

```
function TABLE_DRIVEN (percept)
    persistent: percepts (sequence)
     append percept to percepts
      ACTION → LOOKUP (percepts, table)
       return action.
```

Simple Reflex Agent

→ Simplest kind of agent

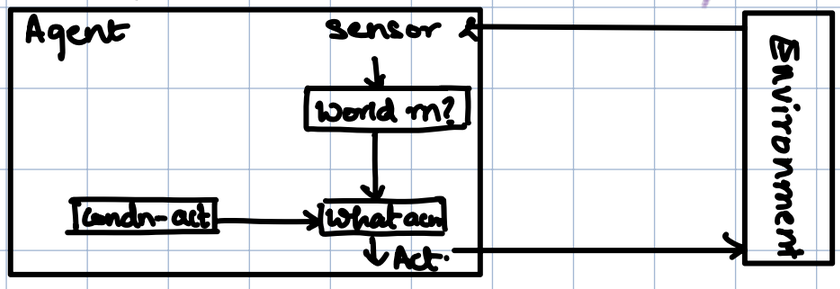
→ Agent choose action based on CURRENT percept & ignores percept history

→ CONDITION-ACTION Rule

if [condi.] then [action]

→ Fully Observable

Diagram



funcn. SRA (percept) returns action

persistent: rules, set condn-action rules

state ← INTERPRET (percept) (function generating an abstracted desc. of current state from percept)

rule ← RULE-MATCH (rules, state) (returns rule that matches state)

action ← rule.ACTION

return action

Problem:

① Very limited intelligence ② No knowledge of non-percept. parts of state

③ Change in environment → rules → update

Eg:

Automatic Door

Percept : Motion Detected

Action : Open Door

Rule : IF motion detected → open door

Thermostat

Percept : Current temp.

Action : Heater ON/OFF

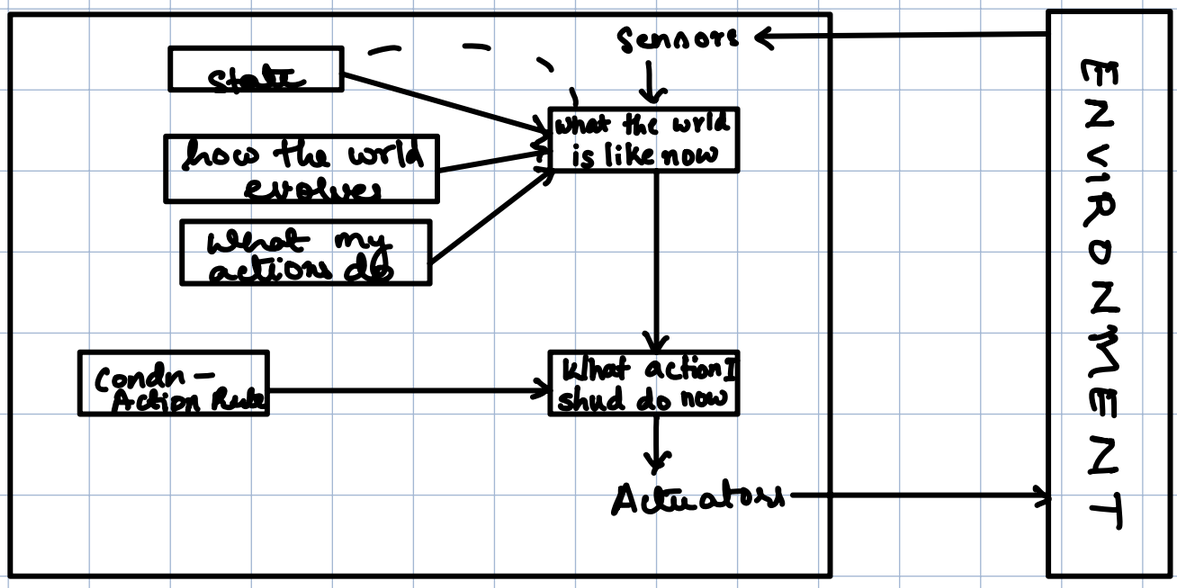
Rule IF temp < 20°C → ON

Model - Reflexive Agent

→ effective way to handle the partial observability, agent keeps track of the world it can't see.

→ Agent keep some internal state (depends on percept history thereby reflects at some unobserved aspects of the current state)

Percieve → Remember → Reason → Act



function MBRA (percept) returns action:

persistant:

state → the agent's perception of world state

model → how world evolves + action conseq.

rules → set of condn–acn ruls

action → the most recent action

new internal description

state ← Update-State (state, model, action, percept)

rules ← Rule-Match (state, rules)

action ← rules.ACTION

return action

Model = internal memory of the environment

stores:

① what the agent has seen before

② How world changes over time

③ How action affects env.

Updating Internal state info →

① How the world evolves independently of agent

② How the agent's own action affect the world

Eg: Robot Vacuum

Components:

1. Sensor	2. Internal State (robot remembers & believes)	3. World Model (affect of act. + how world works)
Dirt	Map of rooms	move fwd → pos change
Proximity	Cleaned v/s uncleaned	bat. dry ↓ → dock
Location	Obstacle of pos.	area clean → no appear
Battery	Battery status	obstacle → cant move

4. Condition Action

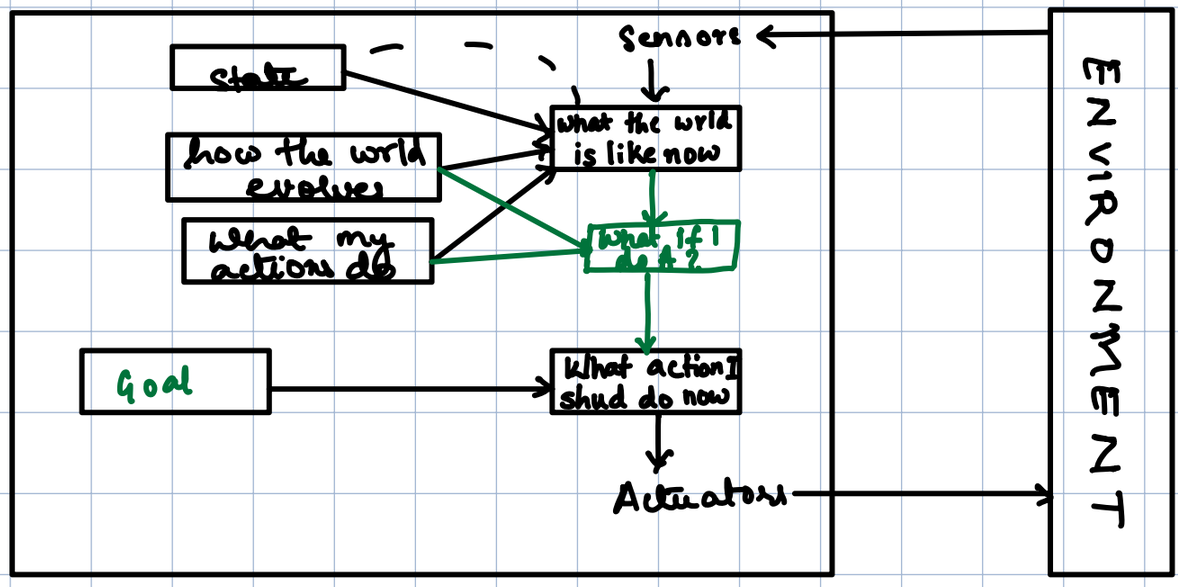
Area dirty — clean

Obstacle ahead — move

Battery low < 20 — dock

Room cleared — move to next room

Goal Based Agent



model based goal agent keeps track of world state as well as a set of goals it trying to achieve

Knowing the current state of the environment is not always enough to decide what to do ⇒ GOAL

eg: Junction

Percieve → Compare Current to Goal → Generate Possible Future Actions → Choose Action closer to goal → Execute → Repeat till goal achieved

Drone – Based Delivery Agent

Sensors:

GPS

Altitude

Obstacle (LiDAR)

Wind

Battery

State:

Current location: Warehouse

Target: Customer's house

Battery: 90%

Weather: Normal

Compare:

Current ≠ Goal

↳ Act

Generate Action

Fly ↓ alt.

Fly ↑ bldgs

Check no fly zone

Planning & Search

Predict pos.

estimate energy

Route A →

Route B →

Select Action Execute Re-Plan (Dynamic) Goal Ach.

Utility-Based Agents

Goals provide binary distinction b/w happy or unhappy state

Eg: Taxi can reach goal

a) Fast + Risky route
b) Medium & Cheap
c) Safe + slow

} All are valid for GBA

→ Models & keeps track of the env., tasks that have research based on - perception, rep, reasoning learning

GOAL

I want to pass the exam

UTILITY ← optimized quality

I want to pass the exam with the highest grade

→ Utility based agents choose action that max. a numerical measure of desirability allowing to handle trade offs & uncertainity better than GBA.

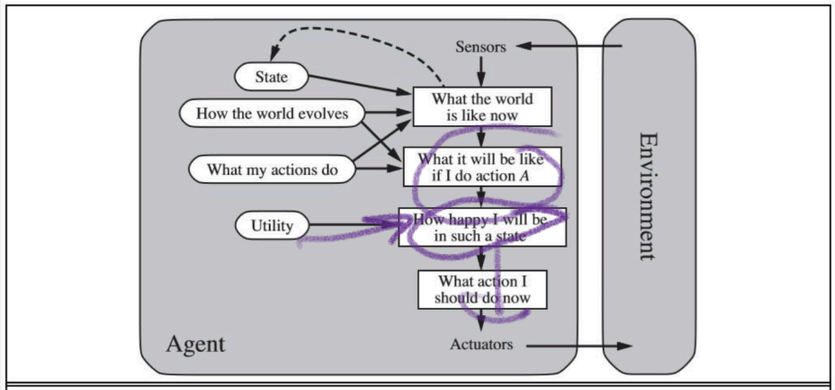


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Learning Agent

↳ AI tool capable of learning from its exp.

↳ Starts with Basic Knowledge → Adapt Auto. through learning, to improve its own performance

Elements:

1) Learning → responsible for improving
2) Performance → selecting external actions
3) Critic → learning element uses feedback from Critic.
   ↳ how performance can be modified to do better in the future.

4) Problem Generator → suggesting action that will lead to new info. exp.

- When you were in school you would do a test and it would be marked the test is the critic. The teacher would mark the test and see what could be improved and instructs you how to do better next time.
- The **teacher** is the **LEARNING ELEMENT** and **you are the PERFORMANCE ELEMENT.**
- **PROBLEM GENERATOR**: - For example coming back to the school analogy, in science with your current knowledge at that time you would not have thought of placing a mass on a spring but the teacher suggested an experiment and you did it and this taught you more and **added to knowledge base.**

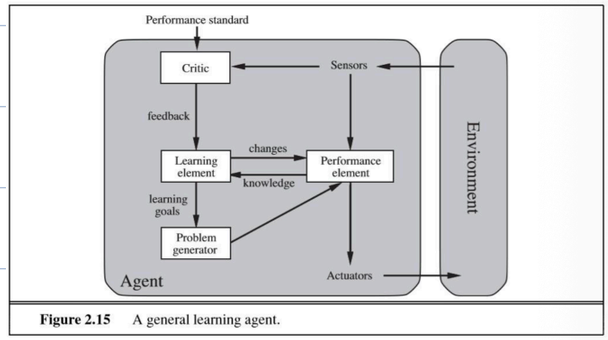
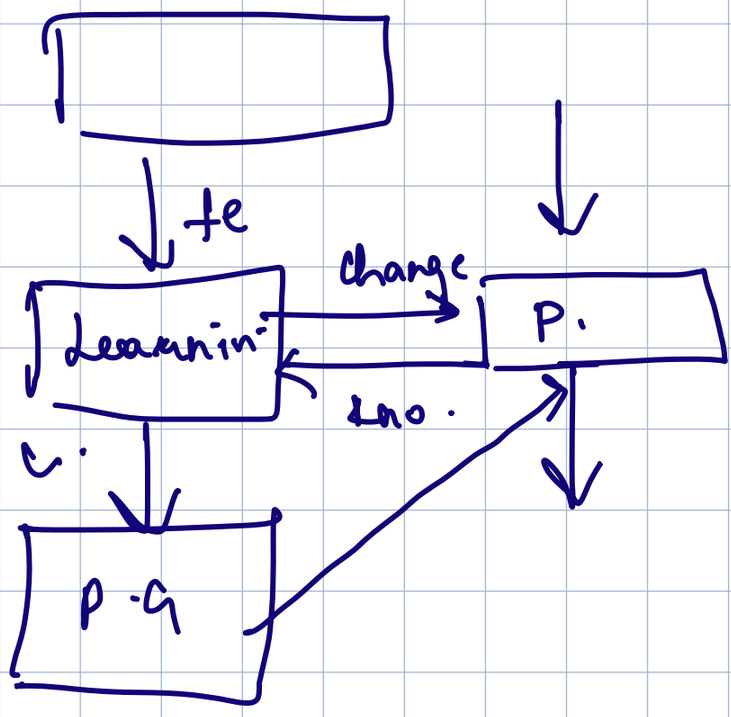


Figure 2.15 A general learning agent.

Learning in intelligent agents can be **summarized** as a process of **modification of each component of the agent to bring the components into closer agreement with the available feedback information, thereby improving the overall performance of the agent.**

Human is an example of a learning agent.

For example, a human can learn to ride a bicycle, even though, at birth, no human possesses this skill.


te
change
Learnin
P.
Ino.
P.q

$\forall x$: person $(x) \rightarrow \exists y$: birthday

$\sim \forall x$: person $(x) \rightarrow \exists y$: birthday (x, y)

$\sim$ person $(x) \vee$ birthday $(x, f(x))$

$\forall x \forall y \forall z$ (goodDog(x) $\wedge$ owns(y, x) $\wedge$ at $(y, z)) \rightarrow$ at (x, z)

]

Al